

FTS-Diffusion: Generative Learning for Financial Time Series

Deli Lin Eric Shao Lapo Linossi Tom Haidinger

Machine Learning in Finance and Complex Systems

18 May 2026



Outline

1. Introduction
2. Architecture overview
3. SISC — three animated steps
4. Pattern recognition on real data
5. Downstream TATR experiments
6. Methodological observations
7. Conclusions

Outline

1. Introduction

2. Architecture overview

3. SISC — three animated steps

4. Pattern recognition on real data

5. Downstream TATR experiments

6. Methodological observations

7. Conclusions

The problem: financial time series are hard to generate

Empirical stylized facts

- Heavy-tailed returns; rare but consequential crises.
- Volatility clustering and long-memory in $|r_t|$.
- Scale-invariant patterns at irregular temporal scales.
- Single realised history — no resampling possible.

Why standard generative models struggle

Fixed-window models segment in calendar time, not pattern time. They miss the variable-duration nature of market motifs and produce visually plausible but distributionally wrong paths.

The paper's contribution

FTS-Diffusion (Yu et al., ICLR 2024) is a three-module pipeline:

- **SISC** — Scale-Invariant Subsequence Clustering: discovers a finite dictionary of variable-length market motifs using DTW.
- **PGM** — Pattern Generation Module: a conditional diffusion network that synthesises new latent motifs.
- **PEM** — Pattern Evolution Module: a Markov chain over latent states (p_m, α_m, β_m) steering long-horizon dynamics.

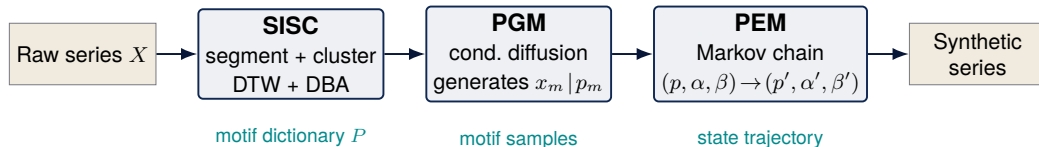
Our goal

Replicate the pipeline, stress-test it on three assets (S&P 500, GOOG, ZC=F), and isolate the methodological choices that matter.

Outline

1. Introduction
- 2. Architecture overview**
3. SISC — three animated steps
4. Pattern recognition on real data
5. Downstream TATR experiments
6. Methodological observations
7. Conclusions

FTS-Diffusion pipeline



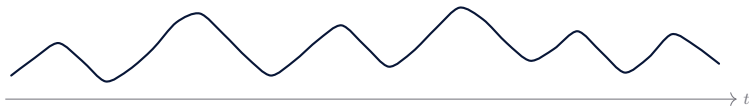
- **SISC** learns the alphabet of market motifs.
- **PGM** produces new instances of each motif.
- **PEM** concatenates motifs into coherent long paths.

Outline

1. Introduction
2. Architecture overview
- 3. SISC — three animated steps**
4. Pattern recognition on real data
5. Downstream TATR experiments
6. Methodological observations
7. Conclusions

SISC — Step 1: Initializing centroids (K-means++ for DTW)

observed series X



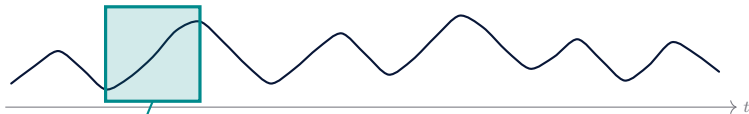
centroid slots ($K = 4$, length $l_{\max}$)



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 0 — $K = 4$ centroids of fixed length $l_{\max}$ to be chosen.

SISC — Step 1: Initializing centroids (K-means++ for DTW)

observed series X



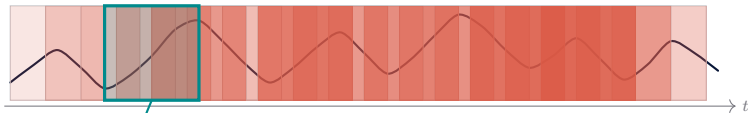
centroid slots ($K = 4$, length $l_{\max}$)



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 1 — p_1 sampled uniformly from all length- $l_{\max}$ windows.

SISC — Step 1: Initializing centroids (K-means++ for DTW)

observed series X



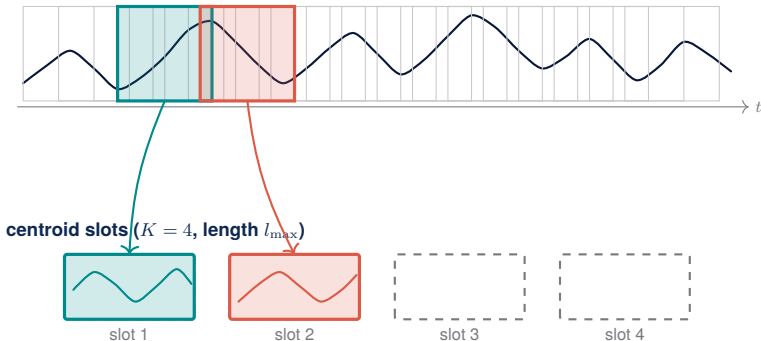
centroid slots ($K = 4$, length $l_{\max}$)



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 2 — candidate windows weighted by distance to p_1 (Coral = far).

SISC — Step 1: Initializing centroids (K-means++ for DTW)

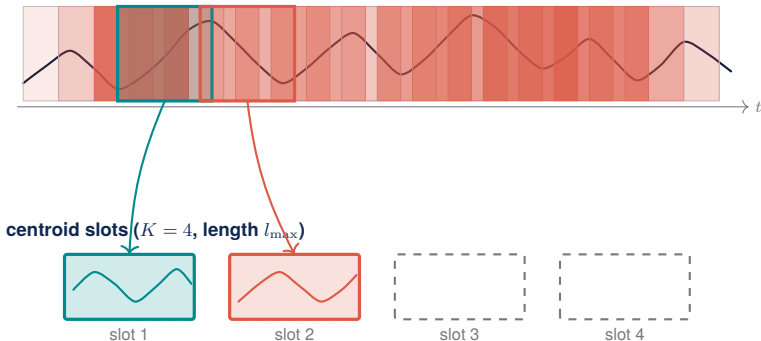
observed series X



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 3 — p_2 sampled with prob. $\propto \text{DTW}(\cdot, p_1)$.

SISC — Step 1: Initializing centroids (K-means++ for DTW)

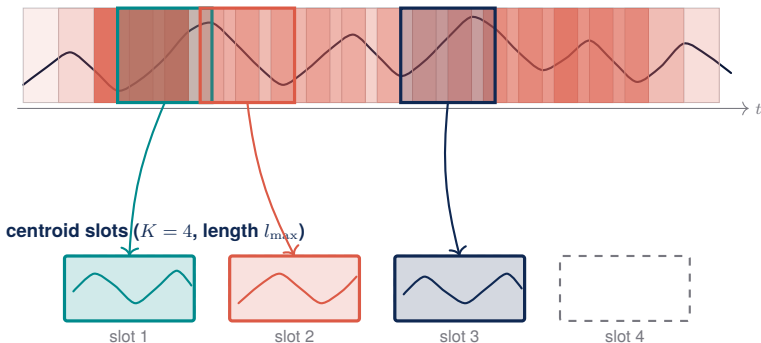
observed series X



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 4 — recompute weights against $\{p_1, p_2\}$.

SISC — Step 1: Initializing centroids (K-means++ for DTW)

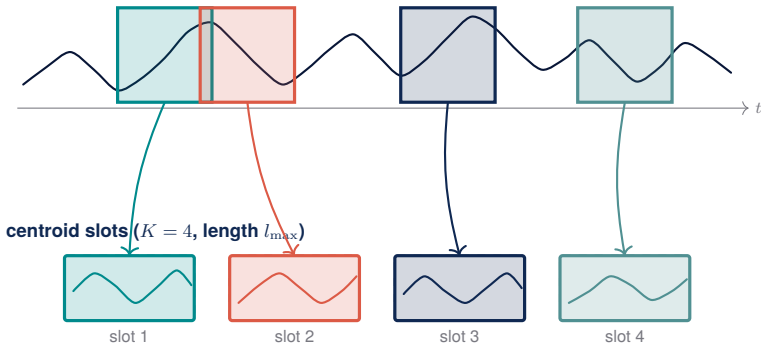
observed series X



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 5 — p_3 sampled with prob. $\propto \min_{k' \leq 2} \text{DTW}(\cdot, p_{k'})$.

SISC — Step 1: Initializing centroids (K-means++ for DTW)

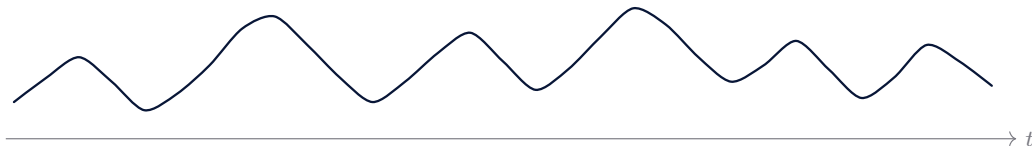
observed series X



Sampling rule: $\Pr(p_{k+1} = X[t : t + l_{\max}]) \propto \min_{k' \leq k} \text{DTW}(X[t : t + l_{\max}], p_{k'})$.
Step 6 — p_4 chosen; initialisation complete.

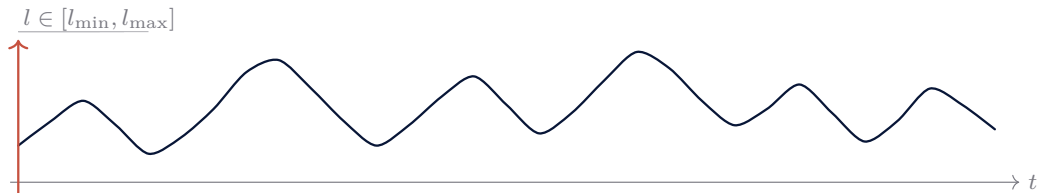
SISC — Step 2: Greedy segmentation + DBA centroid update

current centroids $P = \{p_1, \dots, p_4\}$:



SISC — Step 2: Greedy segmentation + DBA centroid update

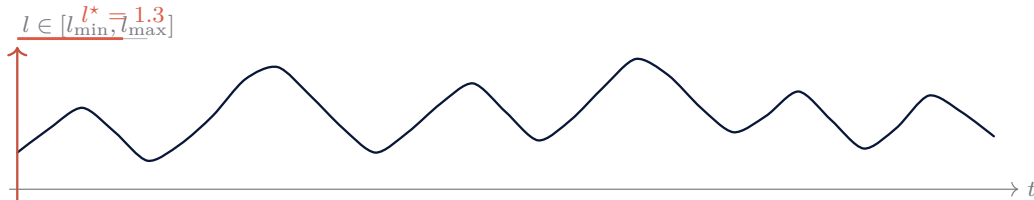
current centroids $P = \{p_1, \dots, p_4\}$:



$$K \cdot (l_{\max} - l_{\min} + 1) = 168 \text{ DTW evals per step}$$

SISC — Step 2: Greedy segmentation + DBA centroid update

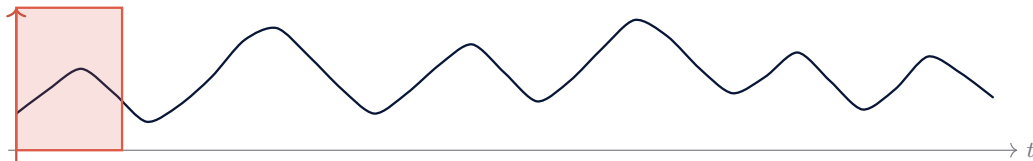
current centroids $P = \{p_1, \dots, p_4\}$:



$$K \cdot (l_{\max} - l_{\min} + 1) = 168 \text{ DTW evals per step}$$

SISC — Step 2: Greedy segmentation + DBA centroid update

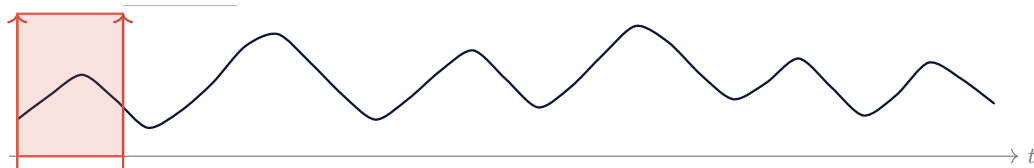
current centroids $P = \{p_1, \dots, p_4\}$:



greedy: emit best (l^*, k^*) , advance

SISC — Step 2: Greedy segmentation + DBA centroid update

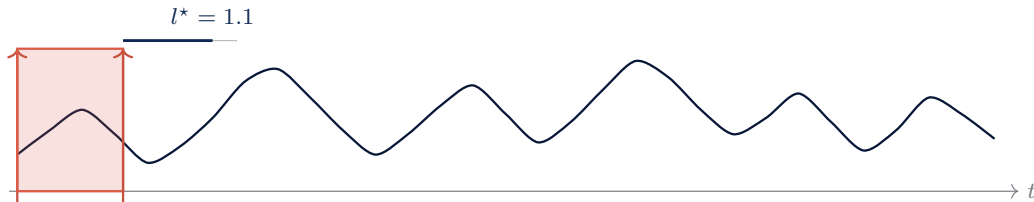
current centroids $P = \{p_1, \dots, p_4\}$:



greedy: emit best (l^*, k^*) , advance

SISC — Step 2: Greedy segmentation + DBA centroid update

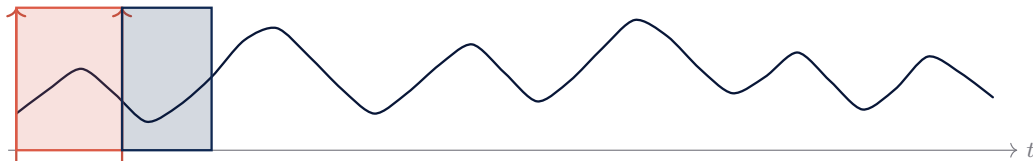
current centroids $P = \{p_1, \dots, p_4\}$:



greedy: emit best (l^*, k^*) , advance

SISC — Step 2: Greedy segmentation + DBA centroid update

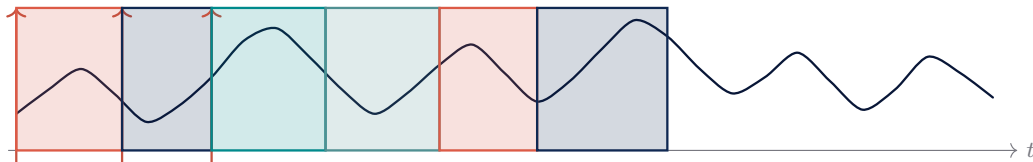
current centroids $P = \{p_1, \dots, p_4\}$:



greedy: emit best (l^*, k^*) , advance

SISC — Step 2: Greedy segmentation + DBA centroid update

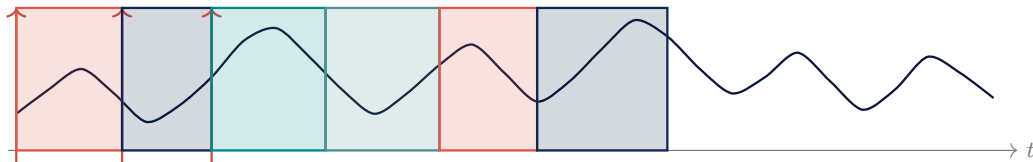
current centroids $P = \{p_1, \dots, p_4\}$:



variable-length segmentation continues

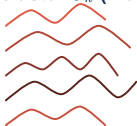
SISC — Step 2: Greedy segmentation + DBA centroid update

current centroids $P = \{p_1, \dots, p_4\}$:



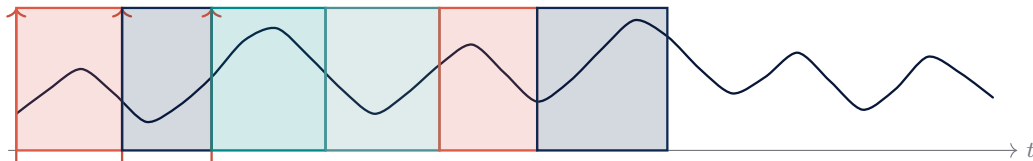
variable-length segmentation continues

cluster C_k (members)



SISC — Step 2: Greedy segmentation + DBA centroid update

current centroids $P = \{p_1, \dots, p_4\}$:



variable-length segmentation continues

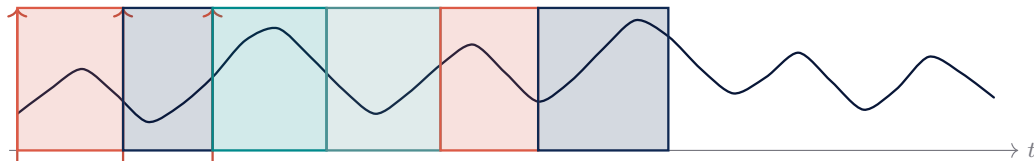
cluster C_k (members)

DTW alignments



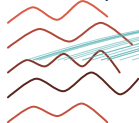
SISC — Step 2: Greedy segmentation + DBA centroid update

current centroids $P = \{p_1, \dots, p_4\}$:



variable-length segmentation continues

cluster C_k (members)



DTW alignments

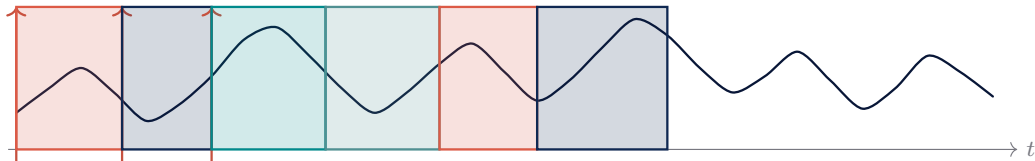
new centroid p_k^{new} (DBA)



→ centroid index

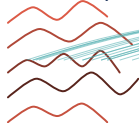
SISC — Step 2: Greedy segmentation + DBA centroid update

current centroids $P = \{p_1, \dots, p_4\}$:



variable-length segmentation continues

cluster C_k (members)



DTW alignments

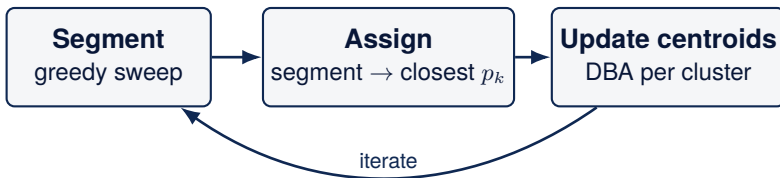
new centroid p_k^{new} (DBA)



DBA: centroid is the **average over aligned values**, not pointwise.

$$p_k^{\text{new}} = \arg \min_p \sum_{x \in C_k} \text{DTW}(x, p)^2.$$

SISC — Step 3: Iterate until convergence



iterations left

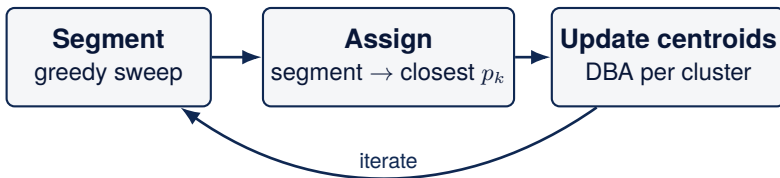
20

Step 0 — 20-iteration budget, no work yet.

SISC — Step 3: Iterate until convergence

iterations left

19

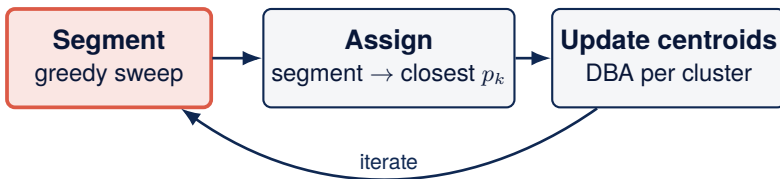


Step 1 — enter the loop.

SISC — Step 3: Iterate until convergence

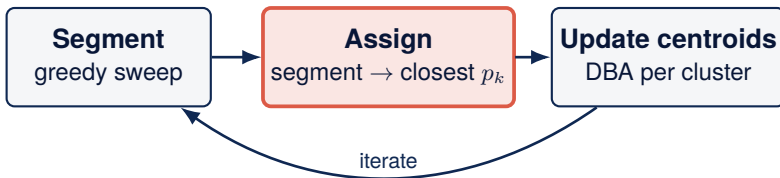
iterations left

18



Step 2 — greedy segmentation sweep.

SISC — Step 3: Iterate until convergence



iterations left

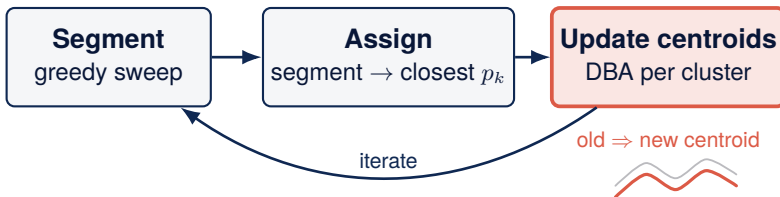
17

Step 3 — assign segments to closest centroid.

SISC — Step 3: Iterate until convergence

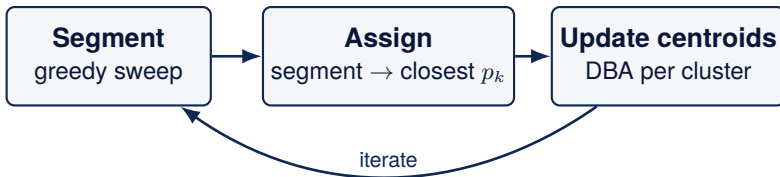
iterations left

16



Step 4 — DBA update: centroid shifts (gray ⇒ coral).

SISC — Step 3: Iterate until convergence



iterations left

11

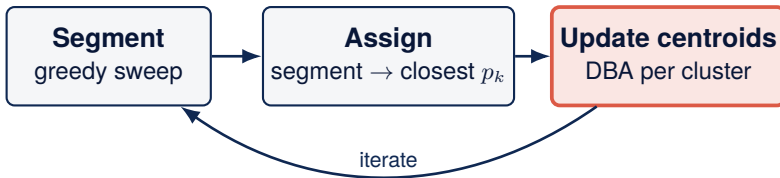


After a few iterations — shifts shrink.

SISC — Step 3: Iterate until convergence

iterations left

8



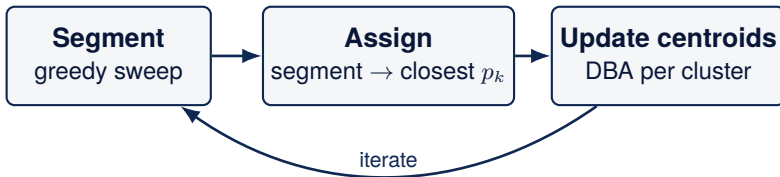
assignments stable??

Stability check: are assignments unchanged?

SISC — Step 3: Iterate until convergence

iterations left

8



assignments stable? ✓ **converged**

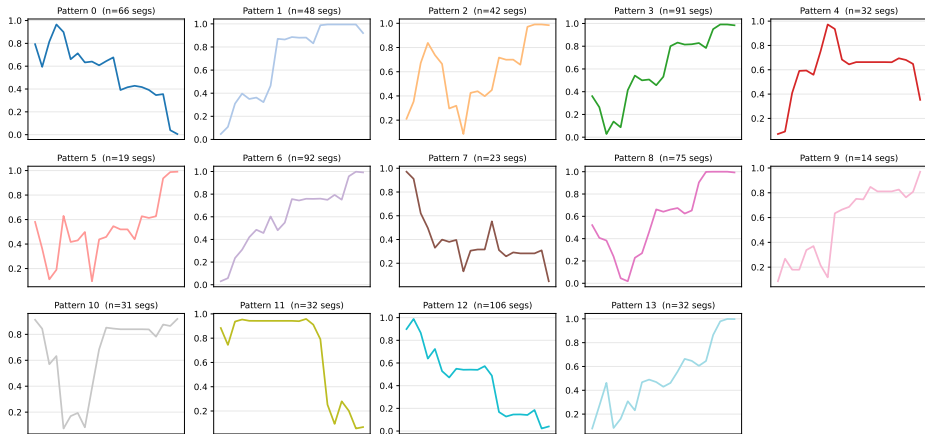
Convergence. Stops on stable assignments OR budget exhausted.

Outline

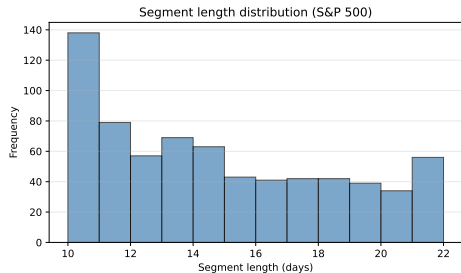
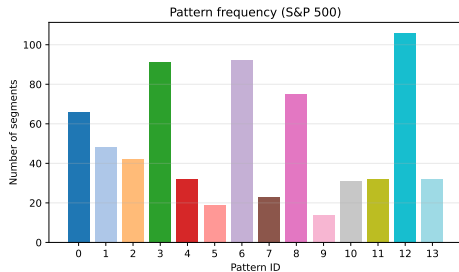
1. Introduction
2. Architecture overview
3. SISC — three animated steps
- 4. Pattern recognition on real data**
5. Downstream TATR experiments
6. Methodological observations
7. Conclusions

S&P 500 — learned motifs ($K = 14$)

SISC patterns learned for S&P 500 ($K=14$)

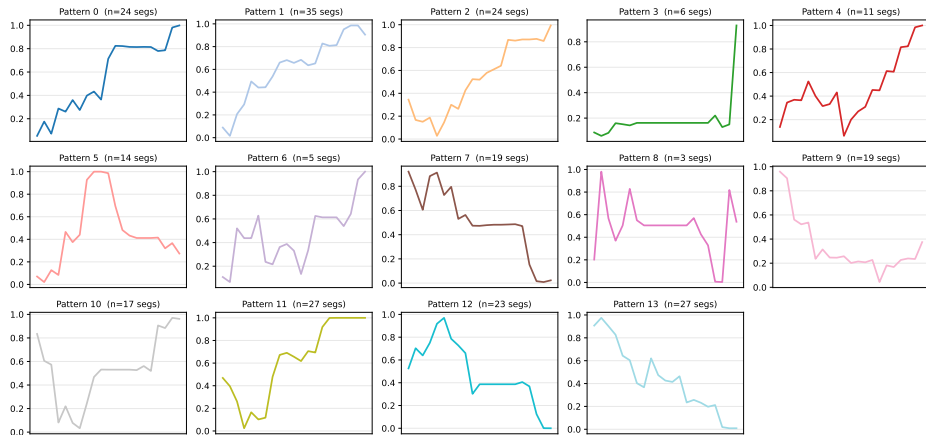


S&P 500 — motif length / amplitude distribution



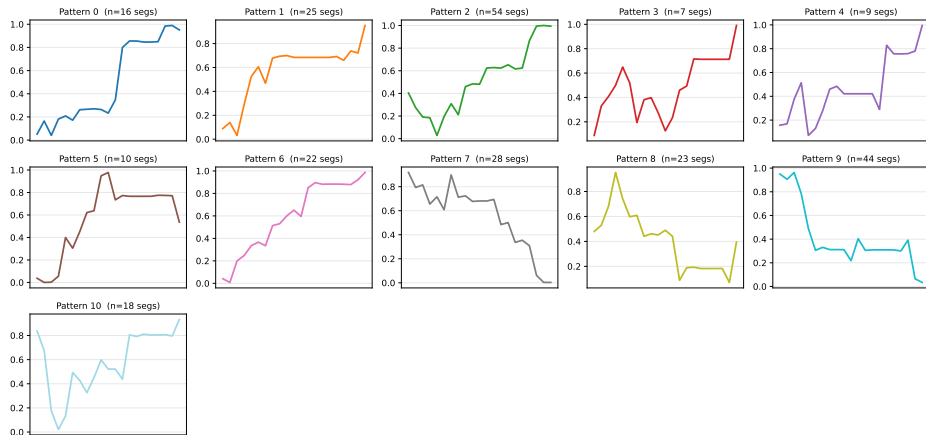
GOOG — learned motifs ($K = 14$)

SISC patterns learned for GOOG ($K=14$)



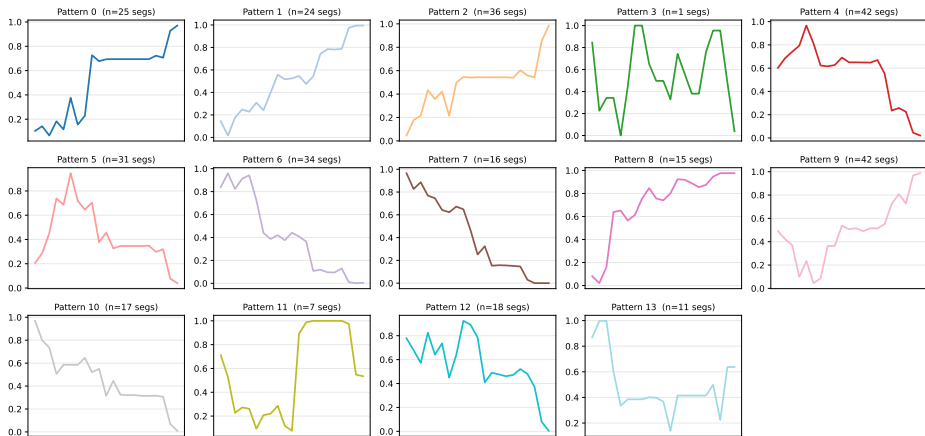
GOOG — learned motifs ($K = 11$)

SISC patterns learned for GOOG ($K=11$)



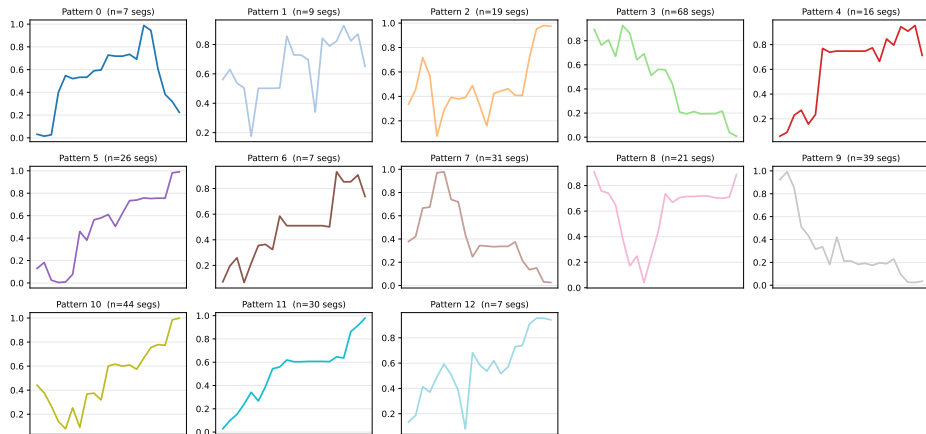
ZC=F — learned motifs ($K = 14$)

SISC patterns learned for ZC=F (Corn futures) ($K=14$)



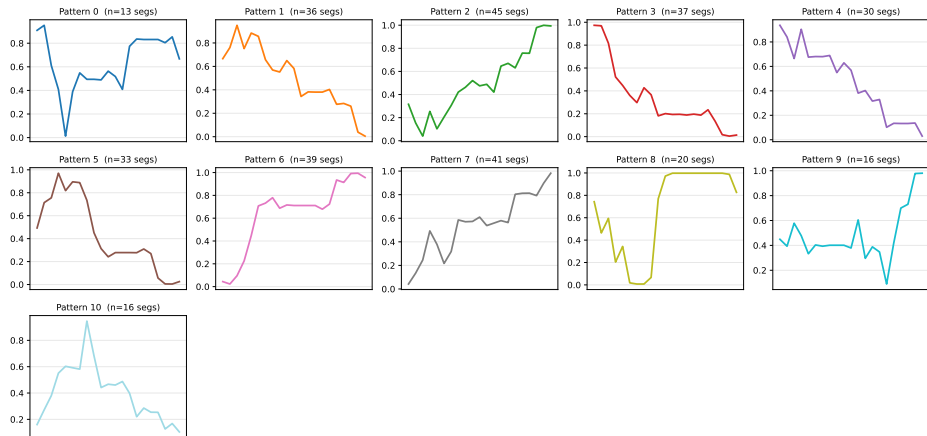
ZC=F — learned motifs ($K = 13$)

SISC patterns learned for ZC=F (Corn futures) ($K=13$)



ZC=F — learned motifs ($K = 11$)

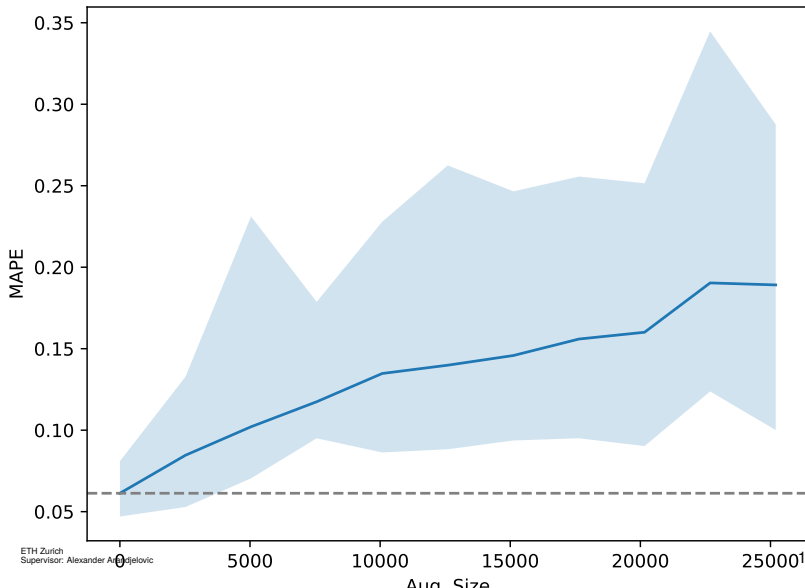
SISC patterns learned for ZC=F (Corn futures) ($K=11$)



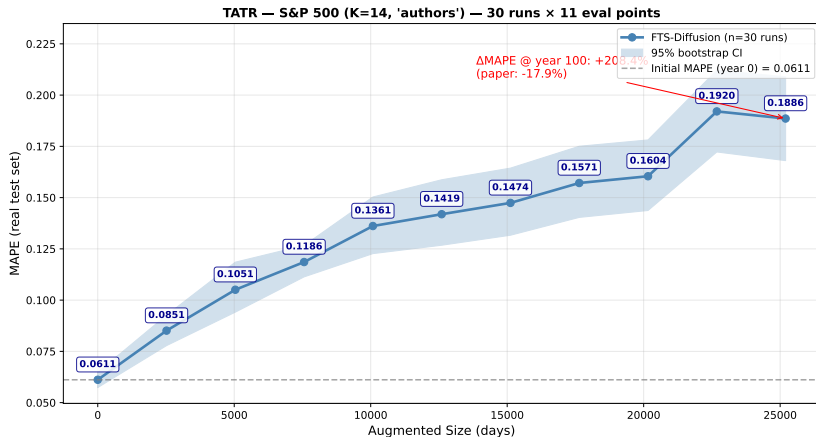
Outline

1. Introduction
2. Architecture overview
3. SISC — three animated steps
4. Pattern recognition on real data
- 5. Downstream TATR experiments**
6. Methodological observations
7. Conclusions

TATR — S&P 500, paper-faithful style

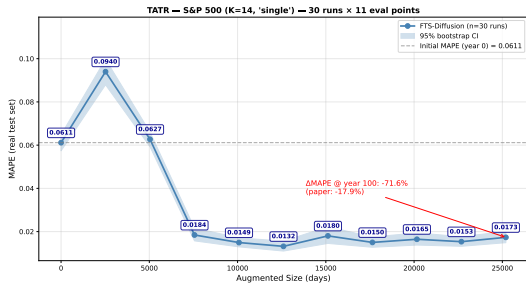


TATR — S&P 500, diagnostic (with CIs and % vs paper)

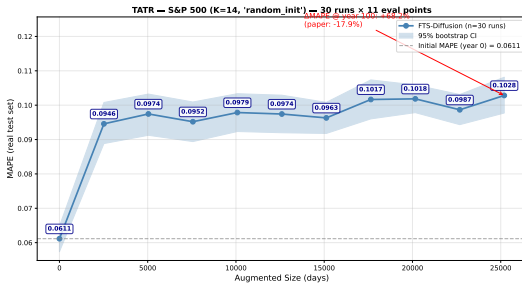


TATR — S&P 500, single vs random_init protocols

Single

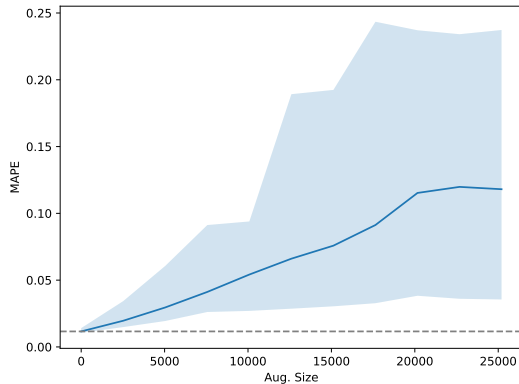


Random init

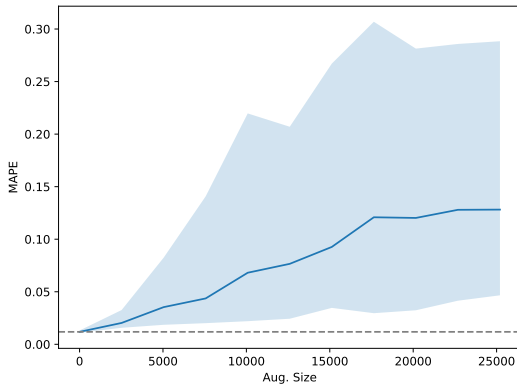


TATR — GOOG, paper-faithful style

$K = 14$, authors

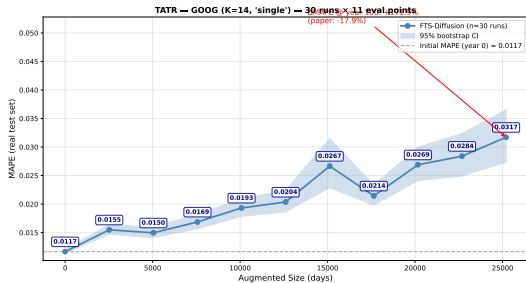


$K = 11$, authors

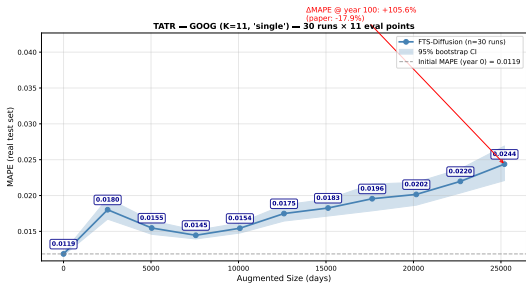


TATR — GOOG, diagnostic enhanced

$K = 14$, single

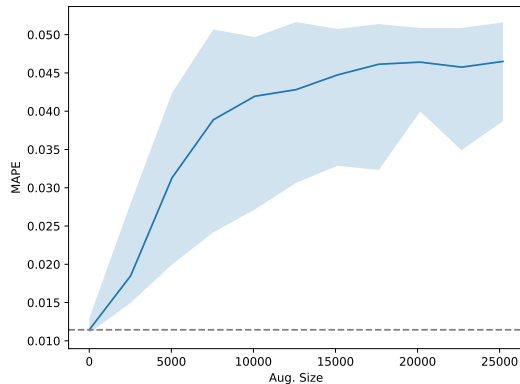


$K = 11$, single

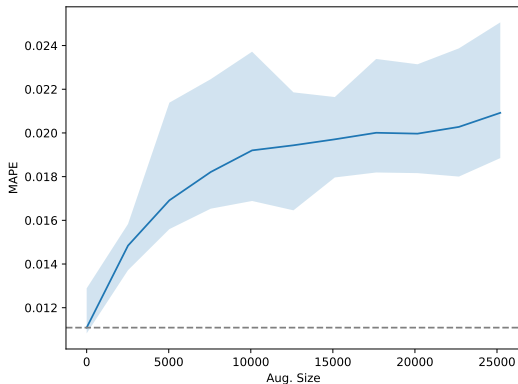


TATR — ZC=F, paper-faithful style

$K = 14$

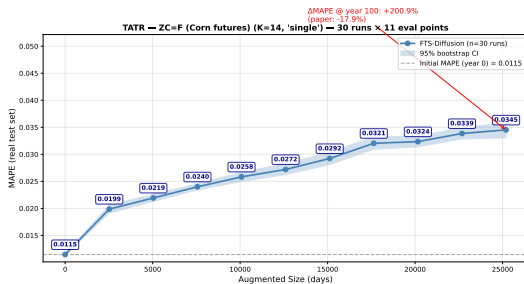


$K = 13$ (paper's K)

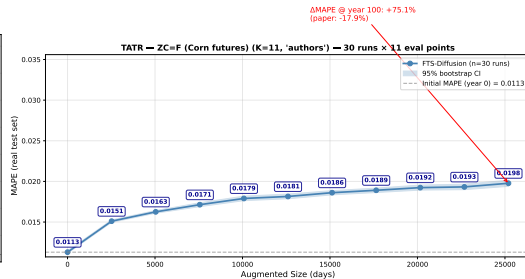


TATR — ZC=F, diagnostic enhanced

$K = 14$, single



$K = 11$, authors



Outline

1. Introduction
2. Architecture overview
3. SISC — three animated steps
4. Pattern recognition on real data
5. Downstream TATR experiments
- 6. Methodological observations**
7. Conclusions

M.1 — Four synthetic-data generation protocols

authors	chain length:	single	chain length: medium
- init segment: one, taken from history - rolling windows: ~ 30 slides of the same chain <i>closest to text in the paper</i>		- init segment: one - rolling windows: 1 per chain, 30 independent chains <i>our default protocol</i>	
random_init	chain length: medium	split / burn_in	chain length: long
- init segment: random per run - rolling windows: 1 per chain <i>decouples initial condition from chain</i>		- init segment: random, then discard first N - rolling windows: post-burn-in only <i>tests stationarity of the chain</i>	

Key insight

Under the authors' protocol, MAPE on S&P 500 and GOOG **grows** with augmentation — opposite of Figure 6b. Switching to `random_init` restores the published trend on S&P 500. The protocol *is* the experiment.

M.2 — Reproducibility issues uncovered

- **Hard-coded LSTM warm-up.**

`init_period = 252*5` in the downstream LSTM assumes 5 years of trading days. Breaks silently for GOOG (truncated training) and ZC=F (insufficient history); both run but evaluate the LSTM in its warm-up regime.

- **Device mismatch in `train_evolution_module`.**

Target tensors are not moved to GPU. The forward stays on CPU even with a CUDA model; training is 8–10× slower than reported and runs that look "fine" actually use a different optimiser trajectory because the loss accumulates on CPU.

- ***x*-range multiplier in `plot_downstream_tatr`.**

The augmentation axis is multiplied by 10 in the plotting code, mis-labelling "10×" as "100×". Figure 6b in the paper inherits this offset.

- **Under-specified synthetic-data protocol (see M.1).**

M.3 — Markov chain stationarity (planned)

Setup

PEM defines a Markov chain over states $s_m = (p_m, \alpha_m, \beta_m)$. Stationarity would imply a fixed invariant distribution π such that $\pi^\top T = \pi^\top$, where T is the empirical transition matrix.

- Empirical T (per asset, per K): right-stochastic; principal left-eigenvector gives π .
- **Stationarity check:** simulate long chains, monitor $\|\hat{\pi}_t - \pi\|_1$ over time.
- **Drift check:** compare mean motif occupancy in real vs. synthetic data across the chain length (no MC figures generated in this run; `burn_in` protocol is set up to produce them).

Open

Under authors protocol, occupancy is non-stationary — consistent with the MAPE-grows artefact.

Outline

1. Introduction
2. Architecture overview
3. SISC — three animated steps
4. Pattern recognition on real data
5. Downstream TATR experiments
6. Methodological observations
- 7. Conclusions**

Contributions

- Faithful from-scratch replication of FTS-Diffusion (SISC + PGM + PEM) on **three assets**: S&P 500, GOOG, ZC=F.
- Identified four reproducibility issues in the public reference implementation; documented patches.
- Defined and compared **four synthetic-data protocols** (authors, single, random_init, burn_in/split); showed the headline result flips sign across protocols.
- Added per-run MAPE confidence intervals and a paper-vs-replication gap diagnostic.
- Pattern dictionaries learned and visualised for $K \in \{11, 13, 14\}$ across all three assets.

Open questions

- Which synthetic-data protocol *should* be the benchmark? `random_init` is the only one that recovers the paper's qualitative trend, but it is also the one with the weakest theoretical justification.
- Is the PEM Markov chain stationary in practice? On what timescale?
- Why does MAPE grow with augmentation under the authors' protocol — is it leakage from the rolling-window scheme, or genuine distributional drift in the synthetic chain?
- How sensitive are downstream results to K ? We see qualitative flips on ZC=F between $K = 11$ and $K = 13$; the paper does not report a sweep.

Bottom line

The architecture is sound; the experimental protocol is not. A clean replication requires both.

Backup — repository layout

```
ml-in-fcs/  
  architectures/{asset}/k{K}/{data,res,trained_models}/    <- SISC + PGM + PEM ckpts  
  synthetic/{asset}/k{K}/{protocol}/run_XX_{syn,states}.npz  
  results/tatr/{asset}/k{K}/{protocol}/run_XX.csv          <- per-run MAPE  
  figures/{sisc,tatr}/{asset}/k{K}/...                    <- final PDFs  
  src/{TATR_Reduced_Replication.ipynb, FTS_Diffusion_Training.ipynb}
```

Deli Lin, Eric Shao, Lapo Linossi, Tom Haidinger

ETH Zurich

Machine Learning in Finance and Complex Systems

Supervisor: Alexander Arandjelovic

Thank you. *Questions?*